MINI PROJECT REPORT

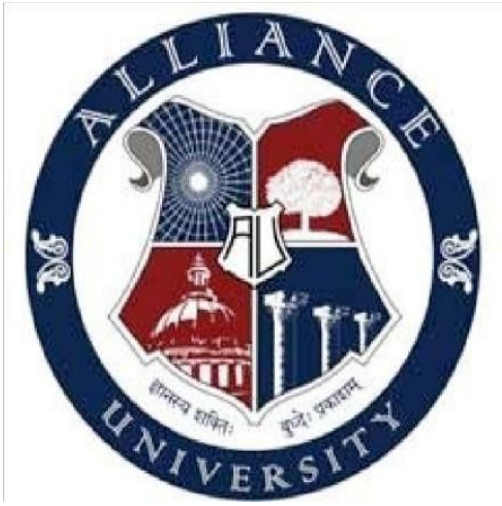
REGULAR EXPRESSION TO FINITE AUTOMATA CONVERSION

Submitted by:
YOUR FULL NAME
YOUR ROLL NUMBER

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
FLAT FACULTY NAME
Assistant Professor, ASAC



Alliance School Of Advance Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on Regular Expression to Finite Automata Conversion (RegexCPU) is submitted by YOUR FULL NAME [Uni. Roll No. YOUR ROLL NUMBER] in partial fulfilment for the award of the degree of Bachelor Of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026. This report embodies the results of original work and studies conducted by students and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

FLAT FACULTY NAME
Assistant Professor



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled REGULAR EXPRESSION TO FINITE AUTOMATA CONVERSION, submitted by me in partial fulfilment of the requirements for the award of the degree of **Bachelor Of Technology in Computer Science and Engineering (CSE)** at **Alliance University**, is a record of my original work carried out under the supervision of **FLAT FACULTY NAME, Assistant Professor, ASAC**.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

YOUR FULL NAME	YOUR ROLL NUMBER	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled REGULAR EXPRESSION TO FINITE AUTOMATA CONVERSION.

First and foremost, I extend my sincere thanks to my mentor and guide, FLAT FACULTY NAME, for the support, insightful feedback, and encouragement throughout the development of this work. Their expertise and commitment to fostering learning provided the foundation for this project.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to implement Thompson's construction, the subset construction and DFA minimisation end to end, and to verify them against a production regex engine.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey.

Thank you.

YOUR FULL NAME

TABLE OF CONTENTS

Acknowledgement.....4

Abstract..... 6

Chapter I- Introduction.....7

Problem Statement.....7

Theory.....7

Chapter II - System Configuration.....9

Algorithm.....9

Source Code.....10

Chapter III - Results..... 12

Chapter IV – Conclusion..... 15

Abstract

Regular expressions and finite automata describe exactly the same class of languages (Kleene's theorem). This report presents a from-scratch implementation of the complete conversion pipeline: a regular expression is parsed with the shunting-yard algorithm, converted into an ε-NFA by Thompson's construction, determinised with the subset construction, completed with a trap state and minimised by partition refinement. The automata are checked on test strings and compared with Python's `re` module on 3,000 random regex/string pairs, with no disagreement.

Experiments confirm the textbook results: the minimal DFA for (a|b)*abb has four states, a Thompson NFA grows linearly with the expression, and the family (a|b)*a(a|b)^n needs 2^(n+1) DFA states, so the DFA grows exponentially while the NFA does not. The automaton produced here is also used by the other two parts of the combined RegexCPU project: it is analysed with Tarjan's SCC algorithm (DAA) and executed on a simulated CPU (COA).

Chapter I- Introduction

Regular expressions are the everyday notation for patterns: validating an e-mail address, tokenising source code, searching logs. Behind most implementations lies a finite automaton. Kleene's theorem states that a language is regular if and only if it is denoted by a regular expression, accepted by a DFA, or accepted by an NFA, and the proof is constructive: it gives algorithms to convert one form into another. Understanding these algorithms is the core of Module 2 of Formal Languages and Automata Theory.

This project implements the constructive proof as working software. The pipeline is: regular expression → postfix form → ε-NFA → DFA → complete DFA → minimal DFA. Each stage is a separate, testable component, and the web application draws every automaton so that the transformations can be inspected visually.

Problem Statement

Library regex engines hide the automaton construction from the user, and hand-drawn conversions in an exam do not scale beyond a few states. The problem is to implement the regular-expression-to-finite-automata conversion faithfully, verify that every stage preserves the language, and study how the sizes of the NFA, DFA and minimal DFA behave. Specific objectives:

- parse regular expressions with union, concatenation, Kleene star, plus, optional, grouping and character classes, and report malformed input clearly;
- build the ε-NFA with Thompson's construction;
- convert it to a DFA with the subset construction using ε-closures;
- complete and minimise the DFA;
- validate all stages against an independent reference (Python `re`) and measure state growth.

Theory

Definitions

A **DFA** is a 5-tuple (Q, Σ, δ, q0, F) with δ: Q × Σ → Q. An **ε-NFA** allows δ: Q × (Σ ∪ {ε}) → 2^Q. Both accept exactly the regular languages. The **ε-closure** of a set of states is the set reachable using only ε-moves.

Thompson's construction

Each operator of the expression is translated into a small NFA fragment with one start and one accepting state. Fragments are glued with ε-edges, so the NFA has at most two states per symbol or operator, i.e. size O(|r|).

Table 1: Thompson construction rules used in nfa.py

Operator	Construction
symbol a	start —a→ accept
r1 r2 (concatenation)	accept(r1) —ε→ start(r2)
r1 | r2 (union)	new start —ε→ both starts; both accepts —ε→ new accept
r* (star)	new start —ε→ start(r) and → new accept; accept(r) —ε→ start(r) and → new accept
r+ (plus)	as star, but no direct ε from new start to new accept
r? (optional)	as star, but no ε-loop back from accept(r) to start(r)

Subset construction

Each DFA state is a set of NFA states. The start state is ε-closure({q0}); for a state S and symbol a, the next state is ε-closure(move(S, a)). A DFA state is accepting if it contains the NFA accepting state. In the worst case the DFA has 2^n states for an n-state NFA; the family (a|b)*a(a|b)^n shows this blow-up is real.

Completion and minimisation

A DFA is made **complete** by adding a trap state that absorbs every missing transition. Two states are equivalent (Myhill–Nerode) if they lead to the same acceptance behaviour on every suffix. **Partition refinement** (Moore) starts with the partition {accepting, non-accepting} and splits blocks whose members move into different blocks on some symbol, until the partition is stable. Each final block becomes one state of the minimal DFA, which is unique up to renaming.

Chapter II - System Configuration

Table 2: Development and execution environment

Item	Configuration
Operating system	Linux 6.18.44-fc-v153
Processor	Intel(R) Xeon(R) Processor @ 2.80GHz
CPU cores	1
Memory (RAM)	3.9 GB
Language / runtime	Python 3.12.3
Libraries	Flask (web app), matplotlib + numpy (graphs); core engines use only the Python standard library
Testing	unittest (28 tests), Python `re` as reference oracle

Components implemented for this report: `regex_parser.py` (tokeniser, explicit concatenation, shunting-yard), `nfa.py` (Thompson's construction, ε-closure, NFA simulation) and `dfa.py` (subset construction, completion, Moore minimisation, trimming and Graphviz export).

Algorithm

Regular expression to postfix

1. Tokenise: literals, operators, escapes; expand a class such as [a-c] into (a|b|c).
2. Insert an explicit concatenation token between adjacent operands, validating operator placement.
3. Run shunting-yard with precedence union < concatenation; postfix operators *, +, ? go straight to the output.

Postfix to ε-NFA, ε-NFA to minimal DFA

4. Scan the postfix string; push a (start, accept) fragment for each symbol; pop one or two fragments for each operator and push the combined fragment.
5. Subset construction: breadth-first search over sets of NFA states, using ε-closure after every move.
6. Add a trap state if any transition is missing (complete DFA).
7. Moore refinement: repeat block ← (block, blocks of successors) until the number of blocks stops growing.
8. Renumber states in breadth-first order so equal languages give identical machines.

Complexity. Parsing and Thompson construction are O(|r|). Subset construction is O(2^n · |Σ|) in the worst case. Moore refinement is O(|Σ| · m^2) for m DFA states (Hopcroft's algorithm would give O(|Σ| · m log m)).

Source Code

Thompson's construction (nfa.py)

```
def thompson(postfix) -> NFA:
    nfa, stack = NFA(), []
    for t in postfix:
        if t.kind == CHAR:
            s, a = nfa.new_state(), nfa.new_state()
            nfa.add(s, t.value, a)
            stack.append((s, a))
        elif t.kind == EPS:
            s, a = nfa.new_state(), nfa.new_state()
            nfa.add(s, EPSILON, a)
            stack.append((s, a))
        elif t.kind == CAT:
            if len(stack) < 2:
                raise RegexError("malformed expression")
            (s2, a2), (s1, a1) = stack.pop(), stack.pop()
            nfa.add(a1, EPSILON, s2)
            stack.append((s1, a2))
        elif t.kind == ALT:
            if len(stack) < 2:
                raise RegexError("malformed expression")
            (s2, a2), (s1, a1) = stack.pop(), stack.pop()
            s, a = nfa.new_state(), nfa.new_state()
            for x in (s1, s2):
                nfa.add(s, EPSILON, x)
            for x in (a1, a2):
                nfa.add(x, EPSILON, a)
            stack.append((s, a))
        elif t.kind in (STAR, PLUS, OPT):
            if not stack:
                raise RegexError("malformed expression")
            s1, a1 = stack.pop()
            s, a = nfa.new_state(), nfa.new_state()
            nfa.add(s, EPSILON, s1)
            nfa.add(a1, EPSILON, a)
            if t.kind in (STAR, OPT):
                nfa.add(s, EPSILON, a)          # skip the operand
            if t.kind in (STAR, PLUS):
                nfa.add(a1, EPSILON, s1)        # loop back
            stack.append((s, a))
    if len(stack) != 1:
        raise RegexError("malformed expression")
    nfa.start, nfa.accept = stack[0]
    return nfa
```

Subset construction (dfa.py)

```
def subset_construction(nfa: NFA) -> DFA:
        alphabet = nfa.alphabet
    start = nfa.eps_closure({nfa.start})
    ids, delta, order = {start: 0}, [{}], deque([start])
    accepting = set()
    while order:
        cur = order.popleft()
        cid = ids[cur]
        if nfa.accept in cur:
            accepting.add(cid)
        for a in alphabet:
            nxt = nfa.eps_closure(nfa.move(cur, a))
            if not nxt:
                continue
            if nxt not in ids:
                ids[nxt] = len(delta)
                delta.append({})
                order.append(nxt)
            delta[cid][a] = ids[nxt]
    return DFA(alphabet, delta, 0, accepting)
```

DFA minimisation (dfa.py)

```
def minimize(dfa: DFA) -> DFA:
        assert dfa.is_complete(), "minimize() needs a complete DFA"
    block = [1 if s in dfa.accepting else 0 for s in range(dfa.n)]
    while True:
        sigs, new_block = {}, [0] * dfa.n
        for s in range(dfa.n):
            sig = (block[s],) + tuple(block[dfa.delta[s][a]] for a in
dfa.alphabet)
            new_block[s] = sigs.setdefault(sig, len(sigs))
        if len(sigs) == len(set(block)):
            block = new_block
            break
        block = new_block
    k = max(block) + 1
    delta = [dict() for _ in range(k)]
    for s in range(dfa.n):
        for a in dfa.alphabet:
            delta[block[s]][a] = block[dfa.delta[s][a]]
    quotient = DFA(list(dfa.alphabet), delta, block[dfa.start],
                   {block[s] for s in dfa.accepting})
    return _renumber(quotient)
```

Chapter III - Results

Worked example: (a|b)*abb

Postfix form: `a b | * a . b . b .`. The ε-NFA has 14 states and 16 edges; the subset construction gives 5 DFA states, which minimise to 4. The result of four states is the well-known answer for this language (strings over {a,b} ending in abb).

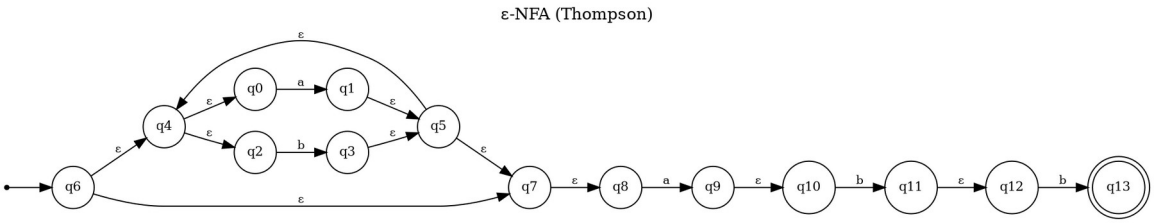


*Figure 1: ε-NFA built by Thompson's construction for (a|b)*abb*

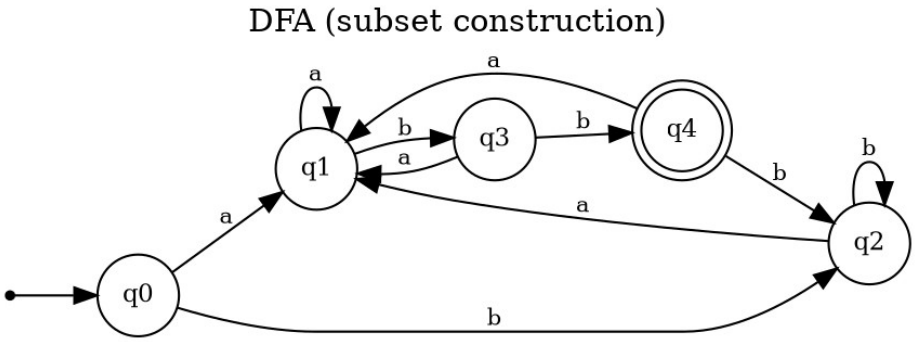


Figure 2: DFA after the subset construction (5 states)

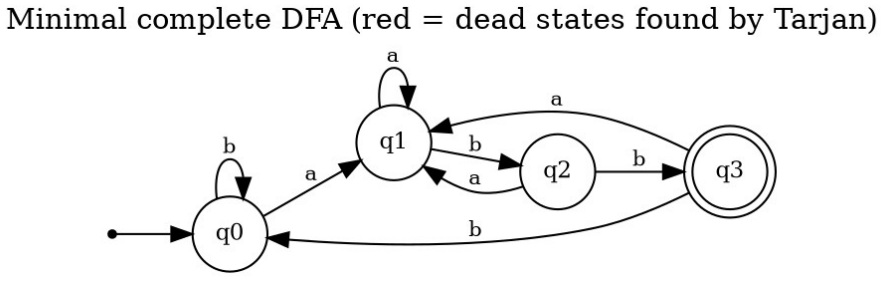


Figure 3: Minimal DFA (4 states)

Test strings

Table 3: Results for pattern `(a|b)*abb`

Input	NFA	DFA	Min DFA	CPU	Agree
`abb`	accept	accept	accept	accept	✔
`aabb`	accept	accept	accept	accept	✔
`babb`	accept	accept	accept	accept	✔
`ababb`	accept	accept	accept	accept	✔
`ab`	reject	reject	reject	reject	✔
`abba`	reject	reject	reject	reject	✔
ε (empty)	reject	reject	reject	reject	✔

A second pattern, `[0-9]+(\.[0-9]+)?` (unsigned decimal number), gives a Thompson NFA of 84 states, a DFA of 22 states and a minimal complete DFA of 5 states (one of them the trap state).

Table 4: Results for pattern `[0-9]+(\.[0-9]+)?`

Input	NFA	DFA	Min DFA	CPU	Agree
`42`	accept	accept	accept	accept	✔
`3.14`	accept	accept	accept	accept	✔
`3.`	reject	reject	reject	reject	✔
`.5`	reject	reject	reject	reject	✔
`1.2.3`	reject	reject	reject	reject	✔
`007`	accept	accept	accept	accept	✔

State growth: NFA versus DFA

Table 5: Size of the automata for (a|b)*a(a|b)^n

n	ε-NFA states	DFA states	Minimal DFA states	Compile time (ms)
1	16	5	4	0.60
2	22	9	8	0.40
3	28	17	16	0.78
4	34	33	32	1.07
5	40	65	64	1.89
6	46	129	128	7.32
7	52	257	256	10.11
8	58	513	512	25.90
9	64	1025	1024	51.58

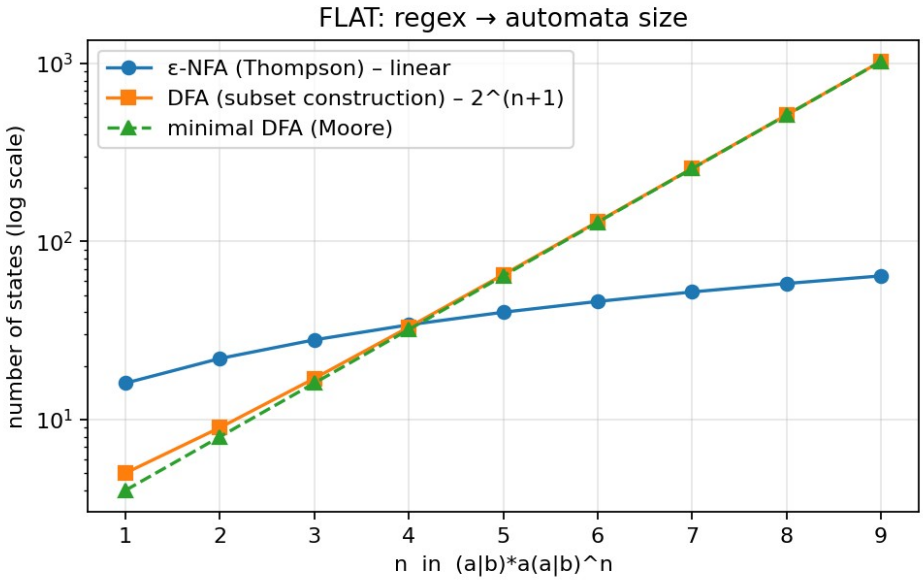


Figure 4: Automaton size against n (logarithmic scale)

The ε-NFA grows by six states for each extra (a|b), i.e. linearly in the expression, while the minimal DFA has exactly 2^(n+1) states: the DFA must remember the last n+1

symbols. This is the practical reason why engines that need a DFA build it lazily or accept exponential cost in rare patterns.

Testing

Table 6: Automated test results (python -m unittest)

Test group	Tests	Result
NFA / DFA / minimisation	5	PASS
CPU, assembler, instruction cycle	7	PASS
End-to-end vs Python `re` (incl. 3,000-case fuzz)	4	PASS
Regex parser (postfix, classes, error handling)	5	PASS
Tarjan & Kosaraju SCC, dead states, finiteness	7	PASS
Total	28	**ALL PASS**

Correctness was established in four ways: (1) hand-checked examples with known minimal sizes; (2) a check that the Thompson NFA never exceeds two states per postfix token; (3) agreement of the NFA, DFA, minimal DFA and the CPU matcher on every test string; (4) a fuzz test generating 250 random regular expressions over {a,b,c} and 12 random strings each (3,000 comparisons) against `re.fullmatch`, which found no disagreement. Malformed expressions such as `(a`, `*a`, `a||b` and `[]` raise a clear error.



Figure 5: Web application: pipeline statistics

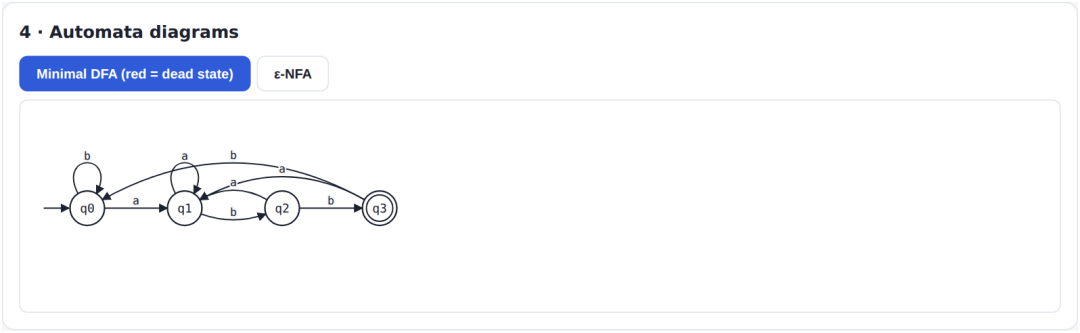


Figure 6: Web application: minimal DFA diagram

Chapter IV – Conclusion

The report demonstrates the constructive content of Kleene's theorem with a complete, tested implementation. Regular expressions with union, concatenation, star, plus, optional and character classes are converted to ε-NFAs, determinised, completed and minimised, and every stage preserved the language on all test inputs, including 3,000 random comparisons with a production regex engine. The experiments reproduce two classical facts: the minimal DFA of (a|b)*abb has four states, and DFA size can be exponential in the size of the expression.

Limitations are the restricted syntax (no wildcard, back-references, anchors or negated classes) and the quadratic Moore minimisation. Future work includes Hopcroft's algorithm, lazy DFA construction, complement and intersection of automata, and counted repetition `{m,n}`.